

P2025R2

Guaranteed copy elision for return variables

Published Proposal, 2021-03-14

Issue Tracking:

[Inline In Spec](#)

Author:

[Anton Zhilin](#)

Audience:

EWG, CWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Source:

<https://github.com/Anton3/cpp-proposals/blob/master/published/p2025r2.bs>

Current:

<https://github.com/Anton3/cpp-proposals/blob/master/published/p2025r2.html>

Abstract

This proposal aims to provide guaranteed copy elision for common cases of local variables being returned from a function, a.k.a. "guaranteed NRVO".

Table of Contents

- 1 **Revision history**
- 2 **Introduction**
- 3 **Motivation**
- 3.1 Construct-cook-return

3.2 Construct-cleanup-return

3.3 Operator rewrites

4 Proposed solution

4.1 Examples

4.1.1 Example 1

4.1.2 Example 2

4.1.3 Example 3

4.1.4 Example 4

4.1.5 Example 5

4.1.6 Example 6

4.1.7 Example 7

4.1.8 Example 8

4.1.9 Example 9

4.1.10 Example 10

4.1.11 Example 11

4.1.12 Example 12

4.1.13 Example 13

4.1.14 Example 14

4.1.15 Example 15

4.1.16 Example 16

4.1.17 Example 17

4.1.18 Example 18

4.1.19 Example 19

4.1.20 Example 20

5 Proposed wording

5.1 Definitions

5.2 The behavior of return variables

5.3 Cross-references

5.4 Returning non-class types

6 Discussion

6.1 Is "return variable" a good term choice?

6.2 Have requirements for copy elision changed?

- 6.2.1 Parentheses are now allowed around the variable name
- 6.2.2 Copy elision is no longer allowed for lambda captures
- 6.2.3 Exception-declarations can introduce return variables
- 6.3 What about trivially copyable temporaries?
- 6.4 What about copy elision for non-class types?
- 6.5 What about the invalidation of optimizations?
- 6.5.1 Complication of escape analysis
- 6.6 Are the proposed changes source or ABI breaking?
- 6.7 What are the costs associated with the proposed changes?

7 Implementation experience

- 7.1 How to deal with exceptions?
- 7.2 An exception-safe implementation strategy
- 7.3 When is copy elision for potential return variables feasible?
- 7.4 A proof of concept implementation in Circle

8 Alternative solutions

- 8.1 Implement similar functionality using existing features
- 8.2 Require an explicit mark for return variables
 - 8.2.1 Attribute
 - 8.2.2 A mark on the variable declaration
 - 8.2.3 A mark on the `return` statement
 - 8.2.4 A syntax that unifies the variable declaration and the `return` statement
 - 8.2.5 Variable declaration at the function definition
 - 8.2.6 A mark on the function definition
 - 8.2.7 A mark on the function declaration
 - 8.2.8 Optional mark
 - 8.2.9 Trivially-copyables: discussion
 - 8.2.10 Guarantee copy elision in more cases: discussion
 - 8.2.11 The comparison table of explicit mark variants
 - 8.2.12 A targeted comparison table of some explicit mark variants

9 Future work

- 9.1 Guarantee some other types of copy elision
- 9.2 Guarantee currently disallowed types of copy elision
- 9.3 Reduce the number of moves performed in other cases

- 9.4 Allow temporaries to be created for types other than trivially-copyables
- 9.5 `std::pin<T>` class
- 9.6 Allow copy elision for complex expressions

10 Acknowledgements

Index

Terms defined by this specification

References

Normative References

Informative References

Issues Index

§ 1. Revision history

R1 (post-Prague):

- The proposal has been rewritten from scratch
- Renamed "named return object" to "return variable"
- Rewrote [§ 5 Proposed wording](#) completely
- Fixed an issue where `return` statements in nested lambdas affected copy elision
- Fixed an issue where the function return object was seemingly constructed and destroyed multiple times
- Fixed the invalidation of optimizations: [§ 6.5 What about the invalidation of optimizations?](#)
- Applied several fixes to copy elision requirements: [§ 6.2 Have requirements for copy elision changed?](#)
- Moved the wording for guaranteed copy elision to [\[class.copy.elision\]](#)
- Used the same terms for copy elision and guaranteed copy elision
- Allowed copy elision for non-class types: [§ 6.4 What about copy elision for non-class types?](#)
- Added some wording for trivially copyable types and expanded [§ 6.3 What about trivially copyable temporaries?](#) section
- Clarified the intent of allowing non-copyable, non-movable types: [§ 4 Proposed solution](#)

- Added the [§ 2 Introduction](#) section
- Expanded the [§ 3 Motivation](#) section
- Added more [§ 4.1 Examples](#)
- Added the [§ 6.5.1 Complication of escape analysis](#) subsection
- Added the [§ 9.5 std::pin<T> class](#) subsection
- Added the [§ 9.4 Allow temporaries to be created for types other than trivially-copyables](#) subsection
- Expanded the [§ 8.2 Require an explicit mark for return variables](#) subsection
- Added the [§ 8.2.8 Optional mark](#) subsection
- Added the [§ 9.6 Allow copy elision for complex expressions](#) subsection
- Added the [§ 7 Implementation experience](#) section

R2:

- Expanded the [§ 8.2 Require an explicit mark for return variables](#) subsection
- Expanded the [§ 6.5.1 Complication of escape analysis](#) subsection
- Minor wording changes

§ 2. Introduction

When a function cannot carry its result in registers, implementations add a hidden parameter containing the address of the caller-owned *return slot* (which is usually an object in the C++ sense). When the function decides to return, the result of the function is stored there. For example, in C++14, 2 copies could be performed for both `foo` and `bar` (assuming `widget` is not trivially copyable):

```

1  widget foo() {      // hidden parameter: widget* r
2      widget x;
3      return x;      // copy
4  }
5  widget bar() {      // hidden parameter: widget* r
6      return widget(); // copy
7  }
8
9  void test() {
10     auto y = foo();  // copy
11     auto z = bar();  // copy

```

`x` or `widget()` was constructed, then copied into the return slot (`*r`), then the return slot was copied into `y` or `z`. The last copy followed from the imperfect pre-C++17 value category design and is irrelevant to our purposes.

The implementation could eliminate some copies under the as-if rule or as allowed by [copy elision](#). The copy in `bar` could be eliminated by Unnamed Return Value Optimization (URVO) and the copy in `foo` — by Named Return Value Optimization (NRVO): `widget()` and `w` would be constructed directly into the return slot. Collectively, both optimizations are sometimes known as RVO; confusingly, URVO is sometimes also called RVO. Note that all of these terms are applied to the implementation and to the physical machine, and are not defined in the Standard.

The updated value category and temporary materialization rules brought by C++17 (informally known as **guaranteed copy elision**) mandated, among everything, that no copies or moves can be made for a returned prvalue expression (except for [trivially copyable](#) class types). This made URVO mandatory in most cases.

In C++20, the only copy allowed in the example is the one at line 3 (actually, `x` is implicitly moved from). Copy elision (NRVO) is allowed there and is routinely performed by most compilers, but is still non-guaranteed, and the `widget` class cannot be non-copyable non-movable. With the proposed wording, `x` is called a [return variable](#), and no copy or move is performed at line 3 — this copy evasion will also informally be called **guaranteed copy elision**.

To differentiate between the two kinds of guaranteed copy elision, it is sometimes useful to use informal terms "guaranteed URVO" and "guaranteed NRVO", meaning the respective language features. Unless otherwise specified, "copy elision" refers to the implementation-defined copy elision provided by [\[class.copy.elision\]](#).

§ 3. Motivation

What follows are the examples where the absence of guaranteed copy elision for [returned](#) variables enforces rewriting the code in a way that is less readable or efficient.

§ 3.1. Construct-cook-return

Sometimes we want to create an object, set it up and return it.

```
widget setup_widget(int x) {  
    widget w;  
    w.set_x(x);  
    return w; // move or copy elision  
}
```

Implementations usually do perform copy elision in such simple cases, but `widget` must be at least movable. This situation is unacceptable in these cases, among others:

- If the setup process includes taking a pointer to `w` and storing it elsewhere
- If `widget` is non-copyable non-movable, because its memory location is critically important to its functionality, e.g. `std::mutex`
- If `widget` is non-copyable, because it manages resources that are not easy to copy, and non-movable, because it does not have an empty state, e.g. `open_file` and `not_null_ptr<T>`

On practice, the workaround can be either:

- Two-stage initialization, where a local variable is constructed in its destination (e.g. using the default constructor) and is then immediately passed to function(s) by reference in order to complete the setup of the object
- Always storing the object on heap, e.g. by returning `std::unique_ptr<widget>` instead of `widget` from factory functions

Both "solutions" are often viewed as anti-patterns. A proper solution should allow for the construct-cook-return pattern, even if a copy or move is not affordable.

§ 3.2. Construct-cleanup-return

With [\[P1144R5\]](#), we may be able to relocate elements out of containers, which should be more efficient:

```
widget widget_owner::pilfer() {  
    widget w = this->relocate_internal();  
    this->cleanup_storage();  
}
```

```
    return w; // move or copy elision
}
```

Unfortunately, such a clean-up work inhibits guaranteed copy elision. This can, however, be worked around using a facility like `scope_success` from [\[P0052R9\]](#):

```
widget widget_owner::pilfer() {
    auto s = scope_success([&]{ this->cleanup_storage(); });
    return this->relocate_internal(); // guaranteed copy elision
}
```

The code rewritten in such a way is less straightforward and contains the potential overhead of `scope_success`.

§ 3.3. Operator rewrites

[\[P1046R2\]](#) proposes automatically generating `operator++(int)` for a type that implements `operator+=(int)`. Its definition would look approximately as follows:

```
T T::operator++(int) {
    T result = *this; // intended copy
    *this += 1;
    return result; // guaranteed copy elision wanted
}
```

In order to deliver on the promise of guaranteed copy elision there, we would have to use the `scope_success` trick described above.

§ 4. Proposed solution

If a returned variable is of the same non-trivially-copyable class type as the return type of the function (ignoring cv-qualification), and all [non-discarded](#) `return` statements in its [potential scope return](#) the variable, guaranteed copy elision is performed. The type of the variable is allowed to be non-copyable, non-movable.

(For the purposes of brevity, the explanation above is not rigorous; see [§ 5 Proposed wording](#) for a rigorous explanation.)

To state the gist of the main requirement even easier (and even less rigorous):

Copy elision is guaranteed for `return x`; if every `return` "seen" by `x` is `return x`;

§ 4.1. Examples

Suppose that `widget` is a class type, is copy-constructible and move-constructible. Unless stated otherwise, references to copy elision utilize [\[class.copy.elision\]/\(1.1\)](#).

Legend:

- ✗ Old
- ✓ Proposed

§ 4.1.1. Example 1

```
1 widget setup_widget(int x) {  
2     return widget(x);  
3 }
```

- ✗✓ No copies or moves needed at line 2 ("guaranteed URVO")

§ 4.1.2. Example 2

```
1 widget setup_widget(int x) {  
2     auto w = widget();  
3     w.set_x(x);  
4     return w;  
5 }
```

- ✗ Move (possibly elided) at line 4
- ✓ No copies or moves needed at line 4 ("guaranteed NRVO")

§ 4.1.3. Example 3

Guaranteed copy elision cannot be performed at line 4, because the `return` statement on line 6, which belongs to the potential scope of `w`, returns non-`w`.

```
1 widget test() {  
2     widget w;  
3     if (...) {  
4         return w;  //!  
5     } else {  
6         return widget();  
7     }  
8 }
```

- ✗✓ Move (possibly elided) at line 4

§ 4.1.4. Example 4

The example above can be "fixed" so that guaranteed copy elision is performed. Now all the `return` statements within the potential scope of `w` (the one on line 4) return `w`.

```
1 widget test() {  
2     if (...) {  
3         widget w;  
4         return w;  
5     } else {  
6         return widget();  
7     }  
8 }
```

- ✗ Move (possibly elided) at line 4
- ✓ No copies or moves needed at line 4 ("guaranteed NRVO")

§ 4.1.5. Example 5

Guaranteed copy elision cannot be performed at line 6, because the `return` statement on line 4, which belongs to the potential scope of `w`, returns non-`w`.

```

1 widget test() {
2     widget w;
3     ...
4     if (...) return {};
5     ...
6     return w;  //!
7 }

```

- ✗ ✓ Move (possibly elided) at line 6

§ 4.1.6. Example 6

The example above can be "fixed" so that guaranteed copy elision is performed. Now all the `return` statements within the potential scope of `w` (the one on line 7) return `w`.

```

1 widget test() {
2     do {
3         widget w;
4         ...
5         if (...) break;
6         ...
7         return w;
8     } while (false);
9     return {};
10 }

```

- ✗ Move (possibly elided) at line 7
- ✓ No copies or moves needed at line 7 ("guaranteed NRVO")

§ 4.1.7. Example 7

Here, the `return` statement at line 2 does not belong to the potential scope of `b` (which starts at line 3), therefore not inhibiting guaranteed copy elision at line 5.

```

1 widget test() {
2     if (...) return {};  //!
3     widget b;
4     ...

```

```
5  return b;
6 }
```

- ✗ Move (possibly elided) at line 5
- ✓ No copies or moves needed at line 5 ("guaranteed NRVO")

§ 4.1.8. Example 8

Here, the `return` statement at line 5 belongs to the potential scope of `one` and thus inhibits guaranteed copy elision for `one`.

```
1 widget test() {
2   widget one;
3   if (toss_a_coin()) return one;  //!
4   widget two;
5   return two;
6 }
```

- ✗ Move (possibly elided) at lines 3 and 5
- ✓ Move (possibly elided) at line 3. No copies or moves needed at line 5 ("guaranteed NRVO")

§ 4.1.9. Example 9

Constructing, setting up and passing an object as a parameter by value using an immediately invoked lambda expression (`p` is directly initialized under the name of `w`).

```
1 void consume_widget(widget p);
2
3 void test(int x) {
4   int y = process(x);
5   consume_widget([&] {
6     auto w = widget(x);
7     w.set_y(y);
8     return w;
9   }());
10 }
```

- ✗ Move (possibly elided) at line 8
- ✓ No copies or moves needed at line 8 ("guaranteed NRVO")

§ 4.1.10. Example 10

If `w` is not returned, it is destroyed, and another `widget` (perhaps, another object named `w`) can take its place.

```

1  widget test() {
2      if (false) {
3          impossible:
4          if (...) return widget();
5      }
6
7      while (...) {
8          widget w;
9          if (...) return w;
10         if (...) break;
11         if (...) continue;
12         if (...) goto impossible;
13         if (...) throw ...;
14         if (...) return w;
15     }
16     return widget();
17 }

```

- ✗ Move (possibly elided) at lines 9 and 14
- ✓ No copies or moves needed at lines 9 and 14 ("guaranteed NRVO")

§ 4.1.11. Example 11

Implementation-wise, `w1`, `w2` and `w3` will occupy the return slot at different times. Wording-wise, if we reach line 10, then `w1` and `w2` are not considered to have ever denoted the result object of the function call; they are considered normal local variables, so the result object of the function is only constructed and destroyed once.

```

1  widget test() {
2      {

```

```

3     {
4         widget w1;
5         if (...) return w1;
6     }
7     widget w2;
8     if (...) return w2;
9 }
10 widget w3;
11 return w3;
12 }

```

- ✗ Move (possibly elided) at lines 5, 8, 11
- ✓ No copies or moves needed at lines 5, 8, 11 ("guaranteed NRVO")

§ 4.1.12. Example 12

Guaranteed copy elision will be unaffected by a nested class, a lambda capture and a discarded `return` statement.

```

1 widget test() {
2     widget w;
3     struct s { widget f() { return widget(); } };
4     auto l = [&w]() { return widget(); }();
5     if constexpr (false) { return widget(); }
6     return w;
7 }

```

- ✗ Move (possibly elided) at line 6
- ✓ No copies or moves needed at line 6 ("guaranteed NRVO")

§ 4.1.13. Example 13

Guaranteed copy elision will be required in constant evaluation context.

```

1 constexpr widget test() {
2     widget x;
3     if (...) return x;
4     widget y;

```

```

5   return y;
6 }
7 constexpr widget z = test();

```

- ✗ Move at lines 3 and 5 (copy elision is not allowed)
- ✓ Move at line 3 (copy elision is not allowed). No copies or moves needed at line 5 ("guaranteed NRVO")

§ 4.1.14. Example 14

```

1 void foo(); // throws a widget
2
3 widget test() {
4     try {
5         foo();
6     } catch (widget w) { //!
7         watch(w);
8         return w;
9     }
10 }

```

- ✗ Copy (possibly elided, [\[class.copy.elision\]/\(1.4\)](#)) at line 6. Move (possibly elided) at line 8
- ✓ Copy (possibly elided, [\[class.copy.elision\]/\(1.4\)](#)) at line 6. No copies or moves needed at line 8 ("guaranteed NRVO")

See also: [§ 6.2.3 Exception-declarations can introduce return variables](#)

§ 4.1.15. Example 15

```

1 widget test() {
2     widget x;
3     if (toss_a_coin()) return (x); //!
4     widget y;
5     return ((y));
6 }

```

- ✗ Move at lines 3 and 5 (copy elision is not allowed, but implementations perform copy elision in this case anyway)
- ✓ Move (possibly elided) at line 3. No copies or moves needed at line 5 ("guaranteed NRVO")

See also: [§ 6.2.1 Parentheses are now allowed around the variable name](#)

§ 4.1.16. Example 16

For trivially copyable types, a copy may still be introduced.

```

1  struct gadget {
2      gadget* p;
3      gadget() : p(this) {}
4  };
5
6  gadget test() {
7      gadget x;
8      return x;  //!
9  }
10 gadget y = test();

```

- ✗ Trivial copy (possibly elided) at line 8. An additional copy may be introduced. It is implementation-defined whether `x` and `y` name the same object and whether `y.p` dangles
- ✓ No copies or moves needed at line 8 ("guaranteed NRVO"). An additional copy may be introduced. It is implementation-defined whether `x` and `y` name the same object and whether `y.p` dangles

See also: [§ 6.3 What about trivially copyable temporaries?](#)

§ 4.1.17. Example 17

For non-class types, a copy may still be introduced.

```

1  using large = std::intmax_t;
2
3  large test() {
4      large x = 42;
5      return x;

```



```

6  }
7
8  large a = test();
9  large b = test() + test();
10 signed char c = test();

```

- ✗ Trivial copy at line 5 (copy elision is not allowed). An additional copy may be introduced. It is implementation-defined whether `x` and `a` name the same object
- ✓ No copies or moves needed at line 5 ("guaranteed NRVO"). An additional copy may be introduced. It is implementation-defined whether `x` and `a` name the same object

See also: [§ 6.4 What about copy elision for non-class types?](#)

§ 4.1.18. Example 18

```

1  template <bool B>
2  widget test() {
3      widget w;
4      if constexpr (B) {
5          if (false) return widget();
6      }
7      return w;
8  }

```

- ✗ Move (possibly elided) at line 7
- ✓ In the instantiation where `B == false`, no copies or moves are needed at line 7 ("guaranteed NRVO"). In the instantiation where `B == true`, move (possibly elided) at line 7

§ 4.1.19. Example 19

```

1  const volatile widget foo() {
2      widget a;
3      a.mutate(); // OK
4      return a;
5  }
6  auto b = foo();
7

```

```

8  widget bar() {
9      if (...) {
10         const widget c;
11         // c.mutate(); // ERROR
12         // const_cast<widget&>(c).mutate(); // UB
13         return c;
14     }
15     volatile widget d;
16     return d; //!
17 }
18 auto e = foo();
19
20 void baz() {
21     // b.mutate(); // ERROR
22     // const_cast<widget&>(b).mutate(); // UB
23     e.mutate(); // OK
24 }

```

- ✗ Move (possibly elided) at lines 4 and 13. Copy at line 16 (copy elision and implicit move are not allowed)
- ✓ No copies or moves needed at lines 4 and 13 ("guaranteed NRVO"). Copy at line 16 (copy elision and implicit move are not allowed)

§ 4.1.20. Example 20

```

1  extern widget x;
2
3  widget test() {
4      widget y;
5      if (&x == &y) { // true
6          throw 0;
7      }
8      return y;
9  }
10
11 widget x = test();

```

- ✗ Move (possibly elided) at line 8
- ✓ No copies or moves needed at line 8 ("guaranteed NRVO")

§ 5. Proposed wording

The wording in this section is relative to WG21 draft [\[N4861\]](#).

§ 5.1. Definitions

Add new sections in [\[class.copy.elision\]](#):

A `return` ([\[stmt.return\]](#)) or `co_return` ([\[stmt.return.coroutine\]](#)) statement *R* **directly observes** a variable *V* if the potential scope ([\[basic.scope\]](#)) of *V* includes *R* and *V* is declared in the body or *parameter-declaration-clause* of the innermost enclosing function or *lambda-expression* of *R*. If, additionally, the operand of *R* is a (possibly parenthesized) *id-expression* designating *V*, then *R* **returns** the variable *V*.

A local variable is called a **potential return variable** when all of the following conditions are satisfied:

- the variable is not explicitly declared `static`, `thread_local`, or `extern`,
- the type of the variable is an object type, is not `volatile`-qualified, and is the same (ignoring cv-qualification) as the return type of the immediately enclosing function or *lambda-expression*, and
- at least one non-discarded ([\[stmt.if\]](#)) return statement [returns](#) the variable. [*Note*: The enclosing function cannot be a coroutine. The variable cannot be a function parameter. — *end note*]

If additionally, all non-discarded return statements, which [directly observe](#) the variable, [return](#) it, then the variable is called a **return variable**.

Note: The definition avoids mentioning the object a [return variable](#) names prematurely.

Note: See also [§ 6.2 Have requirements for copy elision changed?](#), [§ 6.4 What about copy elision for non-class types?](#)

ISSUE 1 Should we say "function or *lambda-expression*", or is it enough to say "function"?

Modify [\[class.copy.elision\]/1](#):

[...] This elision of copy/move operations, called *copy elision*, is permitted in the following circumstances (which may be combined to eliminate multiple copies):

- in a `return` statement in a function with a class return type, when the *expression* is the name of a non-volatile object with automatic storage duration (other than a function parameter or a variable introduced by the *exception-declaration* of a *handler* (`[except.handle]`)) with the same type (ignoring cv-qualification) as the function return type, the copy/move operation can be omitted by constructing the object directly into the function call's return object the implementation can treat a `potential return variable` as a `return variable` [*Note: the variable shall have an eligible copy or move constructor (`[class.copy.ctor]`), unless it is a return variable. — end note*]
- in a *throw-expression* [...]

Modify `[class.copy.elision]/3`:

[...] In the following copy-initialization contexts, a move operation might be used instead of a copy operation:

- If the expression in a `return` (`[stmt.return]`) or `co_return` (`[stmt.return.coroutine]`) statement is a (possibly parenthesized) *id-expression* that names an implicitly movable entity declared in the body or *parameter-declaration-clause* of the innermost enclosing function or *lambda-expression* if a statement `returns` an implicitly movable entity , or
- if the operand of a *throw-expression* [...]

§ 5.2. The behavior of return variables

Add new sections in `[class.copy.elision]`:

A `return variable`'s object occupies the same storage and has the same storage duration as the result object of the function call expression.

If the scope of a [return variable](#) is exited by executing a statement that [returns](#) the variable (and the destruction of local variables is not terminated by an exception), then the variable denotes the result object of the function call expression. [*Note*: If the [return variable](#) is of a trivially copyable type, then a temporary object can be introduced, with a subsequent copy or move ([\[class.temporary\]](#)). In this case the variable shall denote the temporary. — *end note*] The statement that returns the variable performs no copy-initialization ([\[stmt.return\]](#)) and does not cause the destruction of the object ([\[stmt.jump\]](#)). Until the control is transferred out of the function, `const` and `volatile` semantics ([\[dcl.type.cv\]](#)) applied to the object shall correspond to the type of the return variable. [*Example*:

```
class A {
    int x;
    A() = default;
    A(A&&) = delete;
};

A f() {
    A a;          // "a" is a return variable
    a.x = 5;      // OK, a has non-const semantics
    return a;     // OK, no copy-initialization
}

B g() {
    const A b = f();          // "b" names the same object as "a"
    // const_cast<A&>(b).x = 5; // UB, "b" names a const object
}

const A b = f();
```

— *end example*] The destructor for the object is potentially invoked ([\[class.dtor\]](#), [\[except.ctor\]](#)). [*Example*:

```
class A {
    ~A() {}
};

A f() {
    A a;
    return a; // error: destructor of A is private (even though it is never used)
}
```

— *end example*]

After the initialization of the [return variable](#)'s object and before the control is transferred out of the function, if the value of the object or any of its subobjects is accessed through a glvalue that is not obtained, directly or indirectly, from `this` pointer of the object's constructor or from the return variable's name, the value of the object or subobject thus obtained is unspecified.

Note: The object of a [return variable](#) is analogous to an object under construction. Some wording was borrowed from [\[class.ctor\]](#) and [\[class.cdctor\]/2](#).

Note: See also [§ 6.3 What about trivially copyable temporaries?](#), [§ 6.5 What about the invalidation of optimizations?](#)

Modify [\[stmt.jump\]/2](#):

On exit from a scope (however accomplished), objects with automatic storage duration that have been constructed in that scope are destroyed. [. A return variable's object is only destroyed if its scope is exited in a way other than by executing a statement that returns the variable. The objects are destroyed in the reverse order of their construction. \[Note: For temporaries, see \[class.temporary\]. For return variables, see \[class.copy.elision\]. — end note \]](#) Transfer out of a loop, out of a block, or back past an initialized variable with automatic storage duration involves the destruction of objects with automatic storage duration that are in scope at the point transferred from but not at the point transferred to. (See [\[stmt.dcl\]](#) for transfers into blocks). [...]

Modify [\[except.ctor\]/2](#):

Each object with automatic storage duration is destroyed if it has been constructed, but not yet destroyed, since the try block was entered. If an exception is thrown during the destruction of temporaries or local variables for a `return` statement, the destructor for the returned object (if any) is also invoked. [If, additionally, the returned object is named by a return variable, it is only destroyed if the exception propagates out of the scope of the variable.](#) The objects are destroyed in the reverse order of the completion of their construction. [*Example: [...]* — *end example*]

Add an example to [\[except.ctor\]/2](#):

[*Example:*

```
struct A {  
    A() = default;  
    A(A&&) { }  
};  
  
struct X { ~X() noexcept(false) { throw 0; } };  
  
A f() {  
    A a;  
    X x;  
    A b;  
    A c;    // #1  
    A d;  
    return c;    // #2  
}
```

At #1, the [return variable](#) `c` is constructed in the storage for the result object of the function call expression. The local variable `c` does not denote the result object, because the control will exit the scope of `c` by the means of an exception. At #2, the local variable `d` is destroyed, then the local variable `b` is destroyed. Next, the local variable `x` is destroyed, causing stack unwinding, resulting in the destruction of the local variable `c`, followed by the destruction of the local variable `a`. The function call is terminated by the exception.

— *end example*]

§ 5.3. Cross-references

Modify [\[basic.life\]/8](#):

An object *o1* is transparently replaceable by an object *o2* if:

[...]

- neither *o1* nor *o2* is a potentially-overlapping subobject ([intro.object]), and
- either *o1* nor and *o2* are both complete objects, or *o1* and *o2* are denoted by [return variables](#) in the same immediately-enclosing function or *lambda-expression*, or *o1* and *o2* are direct subobjects of objects *p1* and *p2* respectively, and *p1* is transparently replaceable by *p2*.

Modify [\[basic.stc.auto\]/1](#):

Block-scope Local variables not explicitly declared static, thread_local, or extern (other than return variables ([\[class.copy.elision\]](#))) have automatic storage duration. The storage for these entities lasts until the block in which they are created exits.

Modify [\[stmt.return\]/2](#):

[...] A return statement with any other operand shall be used only in a function whose return type is not cv void; the return statement initializes the glvalue result or prvalue result object of the (explicit or implicit) function call by copy-initialization from the operand (except when the statement returns a return variable ([\[class.copy.elision\]](#))). [...]

Modify [\[stmt.dcl\]/2](#):

Variables with automatic storage duration are initialized each time their *declaration-statement* is executed. [*Note*: Variables with automatic storage duration declared in the block are destroyed on exit from the block as described in ([\[stmt.jump\]](#)) . — end note]

Note: The modified sentence currently duplicates the specification in [\[stmt.jump\]/2](#). If the sentence is turned into a reference, it will not have to duplicate the exception for return variables.

Modify [\[class.dtor\]/15](#):

[...] A destructor is potentially invoked if it is invoked or as specified in [expr.new], [stmt.return], [\[class.copy.elision\]](#), [dcl.init.aggr], [class.base.init], and [except.throw]. A program is ill-formed if a destructor that is potentially invoked is deleted or not accessible from the context of the invocation.

§ 5.4. Returning non-class types

Modify [\[class.temporary\]/3](#):

When an object of class type `X` is passed to or returned from a function, if `X` has at least one eligible copy or move constructor ([special]), each such constructor is trivial, and the destructor of `X` is either trivial or deleted, implementations are permitted to create a temporary object to hold the function parameter or result object. The temporary object is constructed from the function argument or return value, respectively, and the function's parameter or return object is initialized as if by using the eligible trivial constructor to copy the temporary (even if that constructor is inaccessible or would not be selected by overload resolution to perform a copy or move of the object). Similarly, when an object of a non-class type is passed to or returned from a function, implementations are permitted to create a temporary object. [*Note*: This latitude is granted to allow objects of class type to be passed to or returned from functions in registers. — *end note*]

Note: CWG2434 ([alt. link](#)) proposes essentially the same change. The wording might be more precise over there.

Add a new section after [\[expr.context\]/2](#):

When a function call prvalue expression of non-class type other than `cv void` is used to compute the value of an operand of a built-in operator, the prvalue result object is a temporary object.

Modify [\[basic.lval\]/5](#):

The result object of a prvalue is the object initialized by the prvalue; a non-discarded prvalue that is used to compute the value of an operand of a built-in operator or a prvalue that has type `cv void` has no result object. [*Note*: Except when the prvalue is the operand of a decltype-specifier, a prvalue of class or array type always has a result object. For a discarded prvalue that has type other than `cv void`, a temporary object is materialized; see [\[expr.context\]](#). — *end note*]

Note: See also [§ 6.4 What about copy elision for non-class types?](#).

§ 6. Discussion

§ 6.1. Is "return variable" a good term choice?

ISSUE 2 "[Return variable](#)" might not be the best term for our purposes.

A previous revision of this proposal (R0) used the term "named return object". That term choice was unfortunate, because it refers to a variable, not to an object. And a variable cannot be "unnamed", so that was excessive.

Some alternative choices:

- result variable
- return object name
- result object name
- transparently returned variable

§ 6.2. Have requirements for copy elision changed?

There are multiple issues with current wording for copy elision. While [\[class.copy.elision\]/3](#) has recently been updated ([\[P0527R1\]](#), [\[P1155R3\]](#), [\[P1825R0\]](#)), [\[class.copy.elision\]/1](#) has not. Proposed wording cleans up those issues.

§ 6.2.1. Parentheses are now allowed around the variable name

See [§ 4.1.15 Example 15](#). `x` is considered a [return variable](#), and `y` a [potential return variable](#).

Meanwhile, at the time of writing, copy elision is not allowed for parenthesized variables.

Implementation divergence has been discovered. Clang and MSVC currently does perform copy elision there, but GCC does not. Consequently, this change may be delivered in a Defect Report.

§ 6.2.2. Copy elision is no longer allowed for lambda captures

In the following case, name lookup for `w` at line 4 finds exactly the outer `w` variable. The `w` expression does satisfy the "name of a non-volatile object with automatic storage duration" condition. Therefore, copy elision is currently allowed for the inner return statement. This seems unintentional; none of the major compilers performs copy elision in this case.

```
1 widget foo() {  
2     widget w;  
3     return [&w] {  
4         return w;  
    }
```

```
5  }();  
6 }
```

- ✗ Move or non-guaranteed copy elision (!) at line 4. No copies or moves needed for `return` at line 3
- ✓ Move at line 4 (copy elision is not allowed). No copies or moves needed for `return` at line 3

This case will no longer be eligible for copy elision under the proposed wording. This change may be delivered in a Defect Report as well.

§ 6.2.3. Exception-declarations can introduce return variables

Note: Guaranteed copy elision will only affect exceptions caught by value ("by copy"). Exceptions caught by reference are not affected.

```
1  struct widget {  
2      widget();  
3      widget(const widget&);  
4      widget(widget&&);  
5  };  
6  
7  void bar();  
8  bool toss_a_coin();  
9  
10 widget foo() {  
11     try {  
12         bar();  
13     } catch (widget w) {           // (1.4)  
14         use(w);  
15         if (toss_a_coin()) return w; // (1.1)  
16     }  
17     return widget();  
18 }
```

Not applying [\[class.copy.elision\]/\(1.4\)](#)

- ✗ At line 13, copy. At line 15, move or non-guaranteed copy elision
- ✓ At line 13, copy. At line 15, guaranteed copy elision

Applying [\[class.copy.elision\]/\(1.4\)](#) (non-guaranteed)

- ✗✓ At line 13, `w` is an alias for the exception in flight. At line 15, copy (copyability is not required if the compiler can prove that `toss_a_coin` never returns `true`) or copy elision ([\[class.copy.elision\]/\(1.1\)](#), highly impractical in this case). The compiler must prove that this application of [\[class.copy.elision\]/\(1.4\)](#) does not change the meaning of the program.

This proposal can inhibit [\[class.copy.elision\]/\(1.4\)](#) in some edge cases for a particular compiler. In return, copy elision is guaranteed consistently for the `return` statement ([\[class.copy.elision\]/\(1.1\)](#)).

Overall, it seems that this is an edge case that can be removed painlessly. To guarantee the absence of copies for the exception object, one should catch the exception by reference, instead of catching by copy and complaining about the copy. On the other hand, when a copy is intended, this proposal ensures that the caught exception is treated as well as any other variable.

ISSUE 3 The previous restriction in this case looks like not-a-defect. Should this change belong to a separate proposal?

§ 6.3. What about trivially copyable temporaries?

According to [\[class.temporary\]](#), the implementation is allowed to create a copy when the object of a trivially copyable type is returned. That is also the case when the copied object participates in (existing or proposed) guaranteed copy elision. If the address of such an object is saved to a pointer variable, the pointer will become dangling on return from the function:

```
class A {
public:
    A* p;
    A() : p(this) {}
};

A existing() {
    return A();
}

A x = existing(); // x.p may be dangling

A* q;
A proposed() {
    A y = A();
    q = &y;
}
```

```
    return y;
}
A z = proposed(); // z.p and q may be dangling
```

Changing [\[class.temporary\]](#) and prohibiting such temporaries would cause ABI breakage, and is infeasible. ABI issues aside, it is not desirable to prohibit optimizations related to liberal treatment of trivially copyable types.

The amount of copying will still be reduced even for those cases. Currently it is implementation-defined whether a copy is elided ([\[class.copy.elision\]/1](#)) and whether a temporary is created ([\[class.temporary\]/3](#)). Depending on that, either 0, 1 or 2 copies may be performed (counting moves as copies). For example:

```
std::pair<int, int> foo() {
    auto a = std::pair(1, 2);
    return a;
}
auto b = foo();
```

Currently, 4 scenarios are possible:

1. Copy elided, no temporary. [a](#) and [b](#) denote the same object. 0 copies
2. Copy elided, temporary. [a](#) denotes the temporary, trivially copied into [b](#). 1 copy
3. Copy not elided, no temporary. [a](#) is a normal local variable, trivially copied into [b](#). 1 copy
4. Copy not elided, temporary. [a](#) is trivially copied into the temporary, which is then trivially copied into [b](#). 2 copies

With this proposal accepted, only scenarios 1 and 2 are possible.

See also: [§ 4.1.16 Example 16](#)

§ 6.4. What about copy elision for non-class types?

Currently, [\[class.copy.elision\]/1](#) disallows copy elision when a variable of a non-class type is returned. Moreover, it is difficult to talk about copy elision for non-class types, because a function call with result of a non-class object type might not have a result object (see [\[expr.prop\]/5](#)).

Meanwhile, there are definitely situations where results of non-class types are passed on stack. An implementation can perform NRVO, so the result gets written directly into the memory location

specified by the caller. For example, on Clang:

```
// substitute with any non-class object type passed on stack,  
// e.g. std::int64_t on a 16-bit system.  
using big_t = _ExtInt(256);  
  
big_t* px = nullptr;  
  
big_t foo() {  
    big_t x = 0;  
    px = &x;  
    return x;  
}  
  
void test() {  
    big_t y = foo();  
    printf("%p %p %d\n", py, &y, py == &y);  
    //=> <addr.> <same addr.> 0  
}
```

While `x` and `y` represent the same object "in the metal", they name different objects in the current C++ sense of "object", thus they compare unequal.

The proposed wording suggests being more honest in this regard by saying that a function with object return type always has a result object, and then by allowing non-class object types to participate in copy elision on equal rights with trivially-copyable objects.

See also: [§ 4.1.17 Example 17](#)

ISSUE 4 Should copy elision for non-class types belong to a separate proposal?

§ 6.5. What about the invalidation of optimizations?

Observe an example:

```
1 struct A {  
2     int x;  
3     A(int x) : x(x) {}  
4     A(const A& o) : x(o.x) {}  
5 };
```

```

6  extern A global;
7
8  A foo() {
9      A local(2);
10     local.x += global.x;
11     return local;
12 }

```

Currently, copy elision is not guaranteed here and cannot make invalid code valid. `global` at line 10 can be assumed to have nothing common with `local`, e.g. `global` cannot be defined as `A global = foo();`. Compilers use this knowledge to optimize `foo` to the equivalent of:

```

A foo() {
    return A(2 + global.x);
}

```

Under this proposal, `local` and `global` can be assumed to be distinct too, for a different reason. Inside `A global = foo();`, `global` and `local` are guaranteed to denote the same object, because `local` is a return variable. Before `foo` returns, the value of `local.x` is accessed through `global.x` that is not obtained from `local`, thus the value of `global.x` is unspecified.

In summary, code which would require invalidation of optimizations for its correctness is kept incorrect.

§ 6.5.1. Complication of escape analysis

Previously, implementations could choose to never perform copy elision ([\[class.copy.elision\]/\(1.1\)](#)) for some non-trivially-copyable class types. For these classes, the implementation could assume that the address of a local variable never escapes:

```

struct widget {
    widget() { }
    widget(widget&&) { }
};

widget foo();    // invisible implementation
widget* bar();  // invisible implementation

void test() {

```

```

    widget x = foo();
    if (&x == bar()) throw "impossible";
}

```

Under the proposed wording, `test` is guaranteed to throw if we define `foo` and `bar` as follows:

```

widget* py;

widget foo() {
    widget y;
    py = &y;
    return y;
}

widget* bar() {
    return py;
}

```

Accounting for such cases can lead to pessimizations for some implementations.

However, I believe that in practice, this is a non-issue. No major compiler currently optimizes the example above assuming that `&x == bar()` is never `true`. Not (only) because the implementors just haven't gotten around to implementing this optimization. No known compiler has explicitly prohibited copy elision at the ABI level. Even though some compiler might have chosen not to perform copy elision in some cases, this has not been enforced ABI-compatible compilers. At this point, enabling this optimization would be an ABI-breaking change. No known compiler is willing to make this decision because of its low cost-benefit ratio.

§ 6.6. Are the proposed changes source or ABI breaking?

Proposed changes *can break* constant expressions that rely on effects of the copy-initialization and destruction that are proposed to be elided. The defect report [\[CWG2278\]](#), requiring that copy elision is not performed in constant expressions, has been presented in March, 2018. However, relying on the effects of copy-initialization and destruction in constant expressions is considered exotic, and real-world code breakage is deemed to be minimal.

The proposal prohibits some corner-case copy elision ([§ 6.2.2 Copy elision is no longer allowed for lambda captures](#)), which was most probably a defect. Note that previously, copy elision could not be relied upon by portable programs.

The proposal allows some new cases of copy elision described in previous sub-sections. Programs that relied on copy elision not being performed there (corner-case scenarios) will no longer be valid.

The proposal is *not* ABI-breaking, because, in all known implementations, whether NRVO is performed for a function does not impact its calling convention.

§ 6.7. What are the costs associated with the proposed changes?

There is no runtime cost associated with the proposed copy elision, because storage for the return object is allocated on stack before the function body starts executing, in all known implementations.

The proposal will make declarations of local variables with automatic storage duration context-dependent: storage of a variable will depend on `return` statements in its potential scope. However, this analysis is local and purely syntactic. The impact on compilation times is thus deemed to be minimal.

Compilers that already perform NRVO will enable it (or at least the required part of it) in all compilation modes. The proposal might even have a positive impact on compilation time, because such implementations will not have to check whether copy-initialization on the return type can be performed.

§ 7. Implementation experience

§ 7.1. How to deal with exceptions?

If the `return rv;` statement is executed, but the destruction of a local variable throws, we may need to destroy the return variable. Some popular compilers, such as GCC and Clang, fail to call the destructor for the return variable in such cases (as of the time of writing) when they choose to perform copy elision:

- [This bug report](#) was filed against Clang in 2012 and still is open
- [This bug report](#) was filed against GCC in 2007 and still is open

Here are some tests that we have to keep in mind when implementing the proposal:

```
#include <cstdio>

struct X {
    char s;
```

```

bool throws;

~X() noexcept(false) {
    printf("~%c\n", s);
    if (throws) throw 0;
}

X(X&& o) = delete;

explicit X(char s, bool throws = false)
    : s(s), throws(throws)
{
    printf("%c\n", s);
}
};

// correct order of destruction: ma
X test1() {
    X m('m', true);
    return X('a');
}

// correct order of destruction: dbmca
X test2() {
    X a('a');
    X m('m', true);
    X b('b');
    X c('c');
    X d('d');
    return c;
}

// correct order of destruction: mab
X test3() {
    X a('a');
    X b('b');
    try {
        X m('m', true);
        return b;
    } catch (...) { }
    return b; // b is returned here
}

```

```

// correct order of destruction if cond:  mbad
// correct order of destruction if !cond: bmcad
X test4(bool cond) {
    X a('a');
    try {
        X m('m', true);
        {
            X b('b');
            if (cond) {
                return b;
            }
        }
        {
            X c('c');
            return c;
        }
    } catch (...) { }
    return X('d');
}

int main() {
    try { test1(); } catch(...) {} puts("");
    try { test2(); } catch(...) {} puts("");
    test3(); puts("");
    test4(true); puts("");
    test4(false); puts("");
}

```

§ 7.2. An exception-safe implementation strategy

All possible "exceptional" cases can be grouped into two categories:

- If the variable, which destruction throws, is defined *inside* the potential scope of the return variable, we need to revert the effects of the `return rv;` statement, and not destroy the variable right away
- If the variable, which destruction throws, is defined *outside* the potential scope of the return variable, we need to destroy the return variable

To deal with the various possible circumstances, the following implementation strategy is suggested:

- There are two flags per function:
 - The "initialized" flag, `true` when the object would be alive even without NRVO
 - The "returned" flag, `true` when we have executed `return rv;` and no destructor has thrown yet
 - Both initially `false`
- After the construction of a return variable:
 - Set "initialized" to `true`
- At the normal destruction of a return variable:
 - Set "initialized" to `false`
 - Only destroy if "returned" is `false`
- At the `return rv;` statement:
 - Set "returned" to `true`
- If the potentially-throwing destructor of a local variable (other than a return variable), which potential scope contains a `return` statement, throws:
 - If "initialized" is `false` and "returned" is `true`, destroy the result slot
 - Set "returned" to `false`
- After the execution of the operand of a `return` statement (other than a `return rv;` statement), but before the destruction of temporaries:
 - Set "returned" to `true`
- In a `return` statement (other than a `return rv;` statement), if the potentially-throwing destructor of a temporary throws:
 - Destroy the return slot
 - Set "returned" to `false`

It is expected that constant propagation eliminates all branches at least for non-exceptional paths. An implementation that uses Table-Driven Exception Handling can instead fold these flags into exception-handling tables, eliminating all overhead for non-exceptional paths.

§ 7.3. When is copy elision for potential return variables feasible?

Extended copy elision (treating a potential return variable as a return variable) does not seem to be easily achievable. For example:

```
bool toss_a_coin() {
    return true;
}

widget test() {
    widget a;
    widget b;
    widget c;
    if (toss_a_coin()) {
        return widget(); // d
    }
    return b;
}
```

If the implementation treats `b` as a return variable, then the destruction shall occur in the order "dcba". (Because `b` does not end up being returned, it shall be treated like a normal local variable.) But that means `b` is destroyed after `d` is constructed, which is not possible. To perform copy elision in this case, the implementation must prove that the destruction of `b` can be moved right before the construction of `d` (and before the destruction of `c`!) under the "as-if" rule.

This only seems feasible in case no non-trivially-destructible variables are declared between the potential return variable `b` and the `return` statement that does not return `b`, and if the operand of the `return` statement is "sufficiently pure". These limiting conditions are satisfied in the following common case:

```
std::vector<int> test() {
    std::vector<int> result;
    fill_vector(result);
    if (something_went_wrong(result)) return {};
    return result;
}
```

§ 7.4. A proof of concept implementation in Circle

The suggested exception-safe algorithm has been implemented by Sean Baxter in [Circle](#) compiler [p2062r0], [build 98](#). For the purposes of this proposal, Circle can be viewed as a Clang fork with several extensions, including, now, guaranteed copy elision for return variables.

The initial testing shows that all the cases are handled correctly. Analysis of the IR and assembly at -O2 shows no sign of the flags - they are eliminated by constant propagation and incorporated into TDEH by the existing passes.

The only limitation of the current implementation is that discarded branches of "if constexpr" can still prevent "guaranteed NRVO" from happening. No non-guaranteed copy elision for potential return variables is provided.

§ 8. Alternative solutions

§ 8.1. Implement similar functionality using existing features

We can implement similar functionality, with cooperation from the returned object type, in some cases.

Suppose the `widget` class defines the following constructor, among others:

```
template <typename... Args, std::invocable<widget&> Func>
widget(Args&&... args, Func&& func)
: widget(std::forward<Args>(args)...)
{ std::invoke(std::forward<Func>(func)), *this); }
```

We can then use it to observe the result object of a prvalue through a reference before returning it:

```
widget setup_widget(int x) {
    int y = process(x);

    return widget(x, [&](widget& w) {
        w.set_y(y);
    });
}
```

However, it requires cooperation from `widget` and breaks when some of its other constructors accept an invocable parameter. We cannot implement this functionality in general.

§ 8.2. Require an explicit mark for return variables

As an alternative, [return variables](#) could require a mark of some sort in order to be eligible for guaranteed copy elision. The advantage of this approach is that the reader of the code is able to see at a glance when guaranteed copy elision is requested, because if unrelated `return` statements block guaranteed copy elision, the mark will trigger a compilation error.

The arguments against requiring an explicit mark are:

- The mark takes space and makes code less elegant
- In the absence of control flow, an explicit mark might seem excessive
- Without the mark, existing code can benefit from guaranteed copy elision, e.g. some function templates that only worked for movable types will start to work for non-movable types

Let's review some of the available options.

§ 8.2.1. Attribute

This is a non-option, because C++ attributes must be ignorable (i.e. removal of an attribute must produce a correct program with similar meaning), but in this case, the attribute would be required for returning variables of non-copyable, non-movable types.

```
std::mutex test() {  
    // std::mutex x;           // error at return  
    std::mutex x [[nrvo]];    // ok  
    return x;  
}
```

The same goes for attributes in other places.

§ 8.2.2. A mark on the variable declaration

We can't use an attribute (see above), so we have to invent some new syntax consisting of punctuation and keywords.

Because we are talking about a way the variable is returned, the most natural choice is to add a `return` declaration modifier somewhere. We must then decide where to place it:

- Before the declarator:

- `return std::mutex x{};` — infeasible, because it's confusingly close to a `return` statement, while having a completely different meaning
- `return -> std::mutex x{};` — seems unnecessarily complex, together with other options involving additional punctuation
- Between the declarator and the initializer:
 - `std::mutex x return {};` — separating the declarator and the initializer seems wrong
- After the initializer:
 - `std::mutex x{} return;` — looks OK

Attributes and multiple variables fit well into the picture:

```
std::mutex x{} return, y{} [[foo]];
```

Here is this syntax in context:

```
std::string test() {
    std::string x return;
    // if (...) return {}; // error
    return x;
}
```

§ 8.2.3. A mark on the `return` statement

Syntax variants:

- Keywords:
 - `return explicit x;`
- Punctuation:
 - `return [x];` — compiler writers might object
 - `return << x;`

Here is this syntax in context:

```
std::string test() {  
    std::string x;  
    // if (...) return {}; // error  
    return explicit x;  
}
```

Let's compare this option against [§ 8.2.2 A mark on the variable declaration](#).

Pros:

- The annotated statement is what's affected by guaranteed copy elision (with the observable behavior that there's no copy or move)
- The already complicated variable declaration is left untouched

Cons:

- When a single variable is returned multiple times, the mark will be duplicated on each of the return statements
- I fail to suggest a syntax as fitting as the `return` keyword on the declaration

§ 8.2.4. A syntax that unifies the variable declaration and the `return` statement

Alias expressions would be a new type of expression. An *alias expression* would accept a prvalue, execute a block, providing that block a "magical" reference to the result object of that prvalue, and the alias expression would itself be a prvalue with the original result object:

```
widget test() {  
    int x = ...;  
    return using (w = widget()) {  
        w.set_x(x);  
        // return {}; // error: a return statement is not allowed here  
        w.set_y(42);  
    };  
}
```

The advantage here is maximum clarity, because it's a special language construct dedicated to this language feature. But alias expressions actually are a special case of *statement expressions* (an

expression that contains statements), and those aren't expected to be accepted any time soon. Additionally, this is arguably too loud of a syntax for a feature that is not groundbreaking.

§ 8.2.5. Variable declaration at the function definition

```
auto test() -> return(widget x{}) {  
    foo(x);  
}
```

This approach has no advantages over [§ 8.2.2 A mark on the variable declaration](#), but is severely limited:

- There can only be one return variable
- This return variable must be initialized at the beginning of the function body

§ 8.2.6. A mark on the function definition

```
std::string test();  
  
std::string test() return {  
    std::string x;  
    // if (...) return {}; // error  
    return x;  
}
```

This mark would require that every `return` statement in the function (which returns some variable?) returns a [return variable](#).

Let's compare this approach to [§ 8.2.2 A mark on the variable declaration](#).

Pros:

- It's not needed to check whether a specific variable is marked, which might increase readability

Cons:

- It introduces a context. In different functions, the same declaration and `return` statement would have slightly different meaning

- In a sufficiently complex function, the restriction on all `return` statements seems unnecessarily limiting

§ 8.2.7. A mark on the function declaration

```
std::string test() return;  
  
std::string test() return {  
    std::string x;  
    // if (...) return {}; // error  
    return x;  
}
```

This option is similar to [§ 8.2.6 A mark on the function definition](#), except that it has a unique advantage compared to all the other solutions. Namely, we can make it affect ABI and enable guaranteed copy elision even for trivially copyable types:

```
int* p;  
  
int test() return {  
    int x = 42;  
    p = &x;  
    return x;  
}  
  
int y = test(); // p points to y
```

With this syntactic option, this is possible, but is it desired? Compilers are allowed to perform extra copies for trivially copyable types, because it results in faster code compared to the return slot approach. In the following example:

```
template <typename T>  
T foo() return {  
    T x = bar();  
    buz(x);  
    return x;  
}
```

Would we want to enforce guaranteed copy elision (at the cost of performance) if `T` turns out to be `int`? In my opinion, we normally wouldn't. This is a fairly niche feature, and when needed, `pin<T>` (§ 9.5 `std::pin<T> class`) can be used to protect against copying.

§ 8.2.8. Optional mark

A compromise approach between "requiring a mark" and "no mark" is to provide "guaranteed NRVO" by default, but allow placing a mark, which checks that the conditions are satisfied, in complex cases.

Such a mark passes the test of ignorability and can be an attribute. However, because the sole meaning of existence of the mark is to guarantee copy elision (as opposed to best-effort, which is provided by default with "optional mark" approach), we will probably want it to be non-attribute. (`constexpr` was made a non-attribute for the same reason.)

The "optional mark" approach works with both [§ 8.2.2 A mark on the variable declaration](#) and [§ 8.2.3 A mark on the return statement](#):

```
widget foo() {
    widget x;
    return x; // no copy-initialization
}

widget bar() {
    widget y return;
    while (...) {
        ...
        if (...) {
            // return baz(); // error
        }
        ...
    }
    return y; // no copy-initialization
}

widget baz() {
    widget z;
    if (...) {
        // return baz(); // error
    }
}
```

```

    }
    return explicit z; // no copy-initialization
}

```

By making the mark optional, we move omission of the mark into code style territory.

§ 8.2.9. Trivially-copyables: discussion

Trivially copyable types are a pain in the neck of guaranteed copy elision. Especially so with an explicit mark: it seems to be enforcing guaranteed copy elision, when in reality it doesn't for trivially copyable types. (Setting aside [§ 8.2.7 A mark on the function declaration.](#))

It seems desirable to prohibit using trivially copyable types with this syntax:

```

int foo() {
    int x = bar();
    return explicit x; // error: int is trivially copyable
}

```

However, this is not really possible for the following reasons:

1. It is implementation-defined whether some standard library types are trivially copyable. For example, `std::optional<int>` is trivially copyable under some, but not all, compilers. As a consequence, it will be implementation-defined whether code like this compiles:

```

std::optional<int> foo() {
    std::optional<int> x = bar();
    return explicit x;
}

```

2. Function templates that wish to make use of the new feature will fail to be compiled for trivially copyable types:

```

template <typename T>
T foo() {
    T x = bar();
    buz(x);
    return explicit x;
}

```

```
foo<int>(); // error
           // even though I probably don't care whether the int is co
```

I suggest that instead of trying to enforce guaranteed copy elision for trivially copyable types, we treat guaranteed copy elision as a matter of *not calling a nontrivial or "heavy" copy or move constructor*. (In case the compiler decides to pass a "heavy" trivially copyable result by stack, it will have the means of doing so without inducing extra copies.)

§ 8.2.10. Guarantee copy elision in more cases: discussion

A frequently used pattern, in which the proposed guaranteed copy elision will **not** apply, is when we initialize the return variable, do something with it, then:

- if all goes well, we return the variable;
- if something fails, we return a newly constructed result representing the failure.

```
std::optional<widget> foo() {
    std::optional<widget> w return;
    fill(w);
    if (...) return {};
    return w;
}
```

Meanwhile, we could, in principle, perform guaranteed copy elision here: if the condition is true, then we could arrange it so that `w` (which is stored as the return object) is destroyed before the operand of `return {};` statement is evaluated.

One issue is related to exceptions:

```
std::optional<widget> foo() {
    std::optional<widget> w return;
    fill(w);
    try {
        if (...) return bar(); // bar throws
    } catch (...) {
        return baz(w);
    }
}
```

```
}  
    return w;  
}
```

We can protect against that by prohibiting the situation when the inner `return` statement is wrapped in a try block, which does not contain the return variable.

More subtle issues arise due to the lifetime of `w` being ended early:

```
std::optional<widget> foo() {  
    std::optional<widget> w return;  
    gadget g{};  
    if (...) return bar();  
    return w;  
}
```

1. `w` could be used inside the `bar()` call
2. `w` could be used inside `g`'s destructor
3. side effects of `w`'s destruction occur before side effects of the `bar()` call

We could guarantee copy elision in this case by requiring yet another mark, with the intent to make the user aware of these potential issues:

```
std::optional<widget> foo() {  
    std::optional<widget> w;  
    gadget g{};  
    if (...) return explicit bar(); // note the additional "explicit"  
    return explicit w;  
}
```

This extension can be pursued in a separate follow-up proposal.

§ 8.2.11. The comparison table of explicit mark variants

	The variable can be declared anywhere in the function; multiple return variables	Allows returning not a return variable on some control flow paths	Allows a copy when returning trivially-copyables	Does not invent an entirely new syntactic construct
<u>At variable declaration, at return statement, optional</u>	✓	✓	✓	✓
<u>At function definition</u>	✓	✗	✓	✓
<u>At function declaration</u>	✓	✗	✗	✓
<u>Variable in the function header</u>	✗	✗	✓	✓
<u>Alias expressions</u>	✓	✓	✓	✗

§ 8.2.12. A targeted comparison table of some explicit mark variants

	The mark is close to where the magic happens	The mark does not have to be repeated in case of multiple return statements	Allows existing code to benefit from guaranteed copy elision	Allows to omit the mark in simple cases
<u>At variable declaration</u>	✗	✓	✗	✗
<u>Optional at variable declaration</u>	✗	✓	✓	✓
<u>At return statement</u>	✓	✗	✗	✗
<u>Optional at return</u>	✓	✗	✓	✓

	The mark is close to where the magic happens	The mark does not have to be repeated in case of multiple <code>return</code> statements	Allows existing code to benefit from guaranteed copy elision	Allows to omit the mark in simple cases
statement				

§ 9. Future work

§ 9.1. Guarantee some other types of copy elision

[\[class.copy.elision\]/1](#) describes 4 cases where copy elision is allowed. Let us review whether it is feasible to guarantee copy elision in those cases:

- (1.1) is feasible to guarantee with the limitations described in this proposal, because such an "optimization" is always correct, does not introduce overhead and does not require non-local reasoning.
- (1.2) leads to an extra allocation in case the control flow escapes the scope before the throw-expression is executed. It would only be feasible to guarantee when the scope contains no other jump statements, and all the functions called are `noexcept`. Those cases are deemed highly specific, but **can** be tackled in a separate proposal.
- (1.3) requires non-local reasoning and is therefore infeasible to guarantee.
- (1.4) requires non-local reasoning and is therefore infeasible to guarantee.

§ 9.2. Guarantee currently disallowed types of copy elision

Requiring copy elision in more cases than is currently allowed by the standard is a breaking change and is out of scope of this proposal. If another proposal that guarantees copy elision in more cases is accepted, those cases could also be reviewed for feasibility of guaranteed copy elision. This proposal will not be influenced by that future work.

§ 9.3. Reduce the number of moves performed in other cases

This proposal belongs to a group of proposals that aim to reduce the number of moves performed in C++ programs. Within that group, there are two subgroups:

- Some proposals allow to replace moves with operations that are yet cheaper than moves (known as relocation or destructive move): [\[N4158\]](#), [\[P0023R0\]](#), [\[P1144R5\]](#).
- Other proposals aim to remove the need for moving altogether. This proposal, as well as [\[P0927R2\]](#), belongs to that group.

The problem solved by the current proposal is orthogonal to the problems dealt with by relocation proposals, as well as to the problem dealt with by P0927R2.

The current proposal combines with [\[P0927R2\]](#) nicely. That proposal requires that the lazy parameter is only used once (and forwarded to another lazy parameter or to its final destination), while in some cases it may be desirable to acquire and use it for some time before forwarding. This proposal would allow to achieve it in a clean way, see the immediately invoked lambda expression example.

The changes proposed by this proposal and [\[P0927R2\]](#), combined, would allow to implement alias expressions (see the corresponding section) without any extra help from the language:

```
template <typename T, invocable<T&> Func>
T also([] -> T value, Func&& func) {
    T computed = value();
    func(computed);
    return computed;
}

void consume_widget(widget);

void test(int x) {
    consume_widget(also(widget(x), [&](auto& w) {
        w.set_x(x);
    }));
}
```

§ 9.4. Allow temporaries to be created for types other than trivially-copyables

Currently, extra copies of the result object are allowed for trivially copyable types, to allow passing those objects in registers, presumably when it is beneficial for performance. A relocation proposal, such as [\[P1144R5\]](#), could allow trivially relocatable types to be treated the same way. If so, then those types will need to be excluded from guaranteed copy elision.

This important change will be source-breaking and will lead to silent UB and bugs if the relocation proposal (with the exemption for trivially relocatable types) is accepted in a later C++ version

compared to this proposal.

§ 9.5. `std::pin<T>` class

For trivially copyable types, copy elision will still be non-guaranteed: the implementation may do a trivial copy to pass the result in registers. Meanwhile, sometimes it is highly desirable to have the guarantee for the absence of copies, e.g. when a pointer to the variable is stored elsewhere. To help in this situation, we may want a non-copyable, non-movable wrapper `pin<T>` which is an aggregate with a single `value` data member. It can be used as follows:

```
struct A {
    A* p;
    constexpr A(int) : p(this) {}
};

constexpr pin<A> foo() {
    pin<A> x{1};
    return x;
}

void test_foo() {
    constexpr auto y = foo();
    static_assert(y.value.p == &y.value); // OK
}

pin<std::string> bar() {
    pin<std::string> std::string y;
    ...
    if (...) return {};
    ...
    return y; // ERROR: y is not a return variable
}
```

The `pin<T>` class can be implemented as follows:

```
struct __pin_non_movable {
    __pin_non_movable& operator=(__pin_non_movable&&) = delete;
};

template <typename T>
```

```

struct __pin_holder {
    T value;
};

template <typename T>
struct pin : __pin_holder<T>, __pin_non_movable { };

```

§ 9.6. Allow copy elision for complex expressions

Can copy elision be allowed in these cases?

```

widget foo() {
    widget x;
    return x += bar();
    return foo(), bar(), x;
    return toss_a_coin() ? foo() : x;
}

```

- For assignment operators, assuming that they return `*this` is infeasible, as voted in Kona 2019 ([P1155R2])
- For a comma operator, it can be overloaded. We could allow it in the non-overloaded case ([CWG2125])
- A ternary operator could be treated as an `if` statement, at which point `x` cannot be a return variable (another `return` in scope), but we could allow it to be a potential return variable, if `x` is not implicitly converted

All in all, it seems that the benefits are not worth the additional complexity.

§ 10. Acknowledgements

Thanks to Agustín Bergé, Arthur O'Dwyer and Krystian Stasiowski, who provided feedback on a draft of this proposal.

Thanks to Jens Maurer, Hubert Tong and everyone else who participated in discussions on the mailing lists and helped polish the proposal.

Thanks to Sean Parent, who implemented the proposal in his Circle compiler and helped with putting together an implementation strategy for exception handling.

Special thanks to Antony Polukhin for championing the proposal.

§ Index

§ Terms defined by this specification

[alias expression](#), in §8.2.4

[potential return variable](#), in §5.1

[return variable](#), in §5.1

[to directly observe a variable](#), in §5.1

[to return a variable](#), in §5.1

§ References

§ Normative References

[N4861]

Richard Smith, Thomas Koepe, Jens Maurer, Dawn Perchik. [Working Draft, Standard for Programming Language C++](#). 8 April 2020. URL: <https://wg21.link/n4861>

§ Informative References

[CWG2125]

Vinny Romano. [Copy elision and comma operator](#). 6 May 2015. extension. URL: <https://wg21.link/cwg2125>

[CWG2278]

Richard Smith. [Copy elision in constant expressions reconsidered](#). 27 June 2016. CD5. URL: <https://wg21.link/cwg2278>

[N4158]

Pablo Halpern. [Destructive Move \(Rev 1\)](#). 12 October 2014. URL: <https://wg21.link/n4158>

[P0023R0]

Denis Bider. [Relocator: Efficiently moving objects](#). 8 April 2016. URL: <https://wg21.link/p0023r0>

[P0052R9]

Peter Sommerlad, Andrew L. Sandoval. [Generic Scope Guard and RAII Wrapper for the Standard Library](#). 3 October 2018. URL: <https://wg21.link/p0052r9>

[P0527R1]

David Stone. [Implicitly move from rvalue references in return statements](#). 8 November 2017. URL: <https://wg21.link/p0527r1>

[P0927R2]

James Dennett, Geoff Romer. [Towards A \(Lazy\) Forwarding Mechanism for C++](#). 5 October 2018. URL: <https://wg21.link/p0927r2>

[P1046R2]

David Stone. [Automatically Generate More Operators](#). 11 January 2020. URL: <https://wg21.link/p1046r2>

[P1144R5]

Arthur O'Dwyer. [Object relocation in terms of move plus destroy](#). 2 March 2020. URL: <https://wg21.link/p1144r5>

[P1155R2]

Arthur O'Dwyer, David Stone. [More implicit moves](#). 19 January 2019. URL: <https://wg21.link/p1155r2>

[P1155R3]

Arthur O'Dwyer, David Stone. [More implicit moves](#). 17 June 2019. URL: <https://wg21.link/p1155r3>

[P1825R0]

David Stone. [Merged wording for P0527R1 and P1155R3](#). 19 July 2019. URL: <https://wg21.link/p1825r0>

[P2062R0]

Daveed Vandevoorde, Wyatt Childers, Andrew Sutton, Faisal Vali. [The Circle Meta-model](#). 13 January 2020. URL: <https://wg21.link/p2062r0>

§ Issues Index

ISSUE 1 Should we say "function or *lambda-expression*", or is it enough to say "function"? [↩](#)

ISSUE 2 "[Return variable](#)" might not be the best term for our purposes. [↩](#)

ISSUE 3 The previous restriction in this case looks like not-a-defect. Should this change belong to a separate proposal? [↩](#)

ISSUE 4 Should copy elision for non-class types belong to a separate proposal? [↩](#)